Implement SQL queries to demonstrate the nested queries in SQL using MySql.
Departments Table:

Create Database University_054;

```
mysql> Create Database University_054;
Query OK, 1 row affected (0.02 sec)
```

Tables:

1) CREATE TABLE Departments (
DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
DeptName VARCHAR(100) UNIQUE NOT NULL,
HOD VARCHAR(100) NOT NULL -- Name of the Head of Department
);

```
mysql> CREATE TABLE Departments_054 (
    ->      DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
    ->      DeptName VARCHAR(100) UNIQUE NOT NULL,
    ->      HOD VARCHAR(100) NOT NULL
    -> );
```

```
mysql> CREATE TABLE Departments_054 (
    ->      DepartmentID INT PRIMARY KEY AUTO_INCREMENT,
    ->      DeptName VARCHAR(100) UNIQUE NOT NULL,
    ->      HOD VARCHAR(100) NOT NULL
    -> );
Query OK, 0 rows affected (0.05 sec)
```

```
INSERT INTO Departments_054 (DeptName, HOD) VALUES
    -> ('Computer Science', 'Dr. Ramesh Sharma'),
    -> ('Electronics', 'Dr. Anjali Verma'),
    -> ('Mechanical', 'Dr. Suresh Kumar'),
    -> ('Civil', 'Dr. Priya Singh');
```

```
mysql> INSERT INTO Departments_054 (DeptName, HOD) VALUES
    -> ('Computer Science', 'Dr. Ramesh Sharma'),
    -> ('Electronics', 'Dr. Anjali Verma'),
    -> ('Mechanical', 'Dr. Suresh Kumar'),
    -> ('Civil', 'Dr. Priya Singh');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

select * from Departments_054;

```
mysql> select * from Departments_054;
+--------------+------------------+--------------------+
| DepartmentID | DeptName         | HOD                |
+--------------+------------------+--------------------+
|            1 | Computer Science | Dr. Ramesh Sharma  |
|            2 | Electronics      | Dr. Anjali Verma   |
|            3 | Mechanical       | Dr. Suresh Kumar   |
|            4 | Civil            | Dr. Priya Singh    |
+--------------+------------------+--------------------+
4 rows in set (0.00 sec)
```

2)

Students Table:

CREATE TABLE Students (
StudentID INT PRIMARY KEY AUTO_INCREMENT,
Name VARCHAR(100) NOT NULL,
Age INT CHECK (Age >= 18), -- Students must be at least 18
DepartmentID INT,
CGPA DECIMAL(3,2) CHECK (CGPA BETWEEN 0 AND 10), -- CGPA should be between 0 and 10
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID) ON DELETE SET NULL
);

```
mysql> CREATE TABLE Students_054(
    -> StudentID INT PRIMARY KEY AUTO_INCREMENT,
    -> Name VARCHAR(100) NOT NULL,
    -> Age INT CHECK (Age >= 18),
    -> DepartmentID INT,
    -> CGPA DECIMAL(3,2) CHECK (CGPA BETWEEN 0 AND 10),
    -> FOREIGN KEY (DepartmentID) REFERENCES Departments_054(DepartmentID) ON DELETE SET NULL
    -> );
```

```
mysql> CREATE TABLE Students_054(
    ->     StudentID INT PRIMARY KEY AUTO_INCREMENT,
    ->     Name VARCHAR(100) NOT NULL,
    ->     Age INT CHECK (Age >= 18),
    ->     DepartmentID INT,
    ->     CGPA DECIMAL(3,2) CHECK (CGPA BETWEEN 0 AND 10),
    ->     FOREIGN KEY (DepartmentID) REFERENCES Departments_054(DepartmentID) ON DELETE SET
 NULL
    -> );
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> INSERT INTO Students_054 (Name, Age, DepartmentID, CGPA) VALUES
    -> ('Aarav Patel', 20, 1, 9.1),
    -> ('Sanya Mehta', 21, 1, 8.5),
    -> ('Rohan Gupta', 22, 2, 7.8),
    -> ('Isha Sharma', 19, 3, 8.9),
    -> ('Vikram Rao', 23, 2, 6.7),
    -> ('Ananya Kapoor', 21, 1, 9.5),
    -> ('Kabir Khan', 22, 4, 7.2);
Query OK, 7 rows affected (0.01 sec)
Records: 7  Duplicates: 0  Warnings: 0
```

```
mysql> select * from Students_054;
+-----------+---------------+------+--------------+------+
| StudentID | Name          | Age  | DepartmentID | CGPA |
+-----------+---------------+------+--------------+------+
|         1 | Aarav Patel   |   20 |            1 | 9.10 |
|         2 | Sanya Mehta   |   21 |            1 | 8.50 |
|         3 | Rohan Gupta   |   22 |            2 | 7.80 |
|         4 | Isha Sharma   |   19 |            3 | 8.90 |
|         5 | Vikram Rao    |   23 |            2 | 6.70 |
|         6 | Ananya Kapoor |   21 |            1 | 9.50 |
|         7 | Kabir Khan    |   22 |            4 | 7.20 |
+-----------+---------------+------+--------------+------+
7 rows in set (0.00 sec)
```

3)
Courses Table:
CREATE TABLE Courses (
CourseID VARCHAR(10) PRIMARY KEY, -- Course codes like CSE101, ECE201
CourseName VARCHAR(100) NOT NULL,
DepartmentID INT,
FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID) ON DELETE CASCADE
);

```
mysql> CREATE TABLE Courses_054 (
    ->     CourseID VARCHAR(10) PRIMARY KEY,
```

-> CourseName VARCHAR(100) NOT NULL,
-> DepartmentID INT,
-> FOREIGN KEY (DepartmentID) REFERENCES Departments_054(DepartmentID) ON DELETE CASCADE
->);

```
mysql> CREATE TABLE Courses_054 (
    ->     CourseID VARCHAR(10) PRIMARY KEY,
    ->     CourseName VARCHAR(100) NOT NULL,
    ->     DepartmentID INT,
    ->     FOREIGN KEY (DepartmentID) REFERENCES Departments_054(DepartmentID) ON DELETE
CASCADE
    -> );
Query OK, 0 rows affected (0.07 sec)
```

```
mysql> select * from Students_054;
+-----------+---------------+------+--------------+------+
| StudentID | Name          | Age  | DepartmentID | CGPA |
+-----------+---------------+------+--------------+------+
|         1 | Aarav Patel   |   20 |            1 | 9.10 |
|         2 | Sanya Mehta   |   21 |            1 | 8.50 |
|         3 | Rohan Gupta   |   22 |            2 | 7.80 |
|         4 | Isha Sharma   |   19 |            3 | 8.90 |
|         5 | Vikram Rao    |   23 |            2 | 6.70 |
|         6 | Ananya Kapoor |   21 |            1 | 9.50 |
|         7 | Kabir Khan    |   22 |            4 | 7.20 |
+-----------+---------------+------+--------------+------+
7 rows in set (0.00 sec)
```


```
mysql> select * from Students_054;
+-----------+---------------+------+--------------+------+
| StudentID | Name          | Age  | DepartmentID | CGPA |
+-----------+---------------+------+--------------+------+
|         1 | Aarav Patel   |   20 |            1 | 9.10 |
|         2 | Sanya Mehta   |   21 |            1 | 8.50 |
|         3 | Rohan Gupta   |   22 |            2 | 7.80 |
|         4 | Isha Sharma   |   19 |            3 | 8.90 |
|         5 | Vikram Rao    |   23 |            2 | 6.70 |
|         6 | Ananya Kapoor |   21 |            1 | 9.50 |
|         7 | Kabir Khan    |   22 |            4 | 7.20 |
+-----------+---------------+------+--------------+------+
7 rows in set (0.00 sec)
```


4)
Enrollments Table:

CREATE TABLE Enrollments (
EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
StudentID INT,

CourseID VARCHAR(10),
Marks INT CHECK (Marks BETWEEN 0 AND 100), -- Marks should be between 0 and 100
FOREIGN KEY (StudentID) REFERENCES Students(StudentID),
FOREIGN KEY (CourseID) REFERENCES Courses(CourseID),
UNIQUE (StudentID, CourseID) -- A student cannot enroll in the same course more than once
);

```
CREATE TABLE Enrollments_054(
    ->      EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
    ->      StudentID INT,
    ->      CourseID VARCHAR(10),
    ->      Marks INT CHECK (Marks BETWEEN 0 AND 100),
    ->      FOREIGN KEY (StudentID) REFERENCES Students_054(StudentID),
    ->      FOREIGN KEY (CourseID) REFERENCES Courses_054(CourseID),
    ->      UNIQUE (StudentID, CourseID)
    -> );
```

```
mysql> CREATE TABLE Enrollments_054(
    ->      EnrollmentID INT PRIMARY KEY AUTO_INCREMENT,
    ->      StudentID INT,
    ->      CourseID VARCHAR(10),
    ->      Marks INT CHECK (Marks BETWEEN 0 AND 100),
    ->      FOREIGN KEY (StudentID) REFERENCES Students_054(StudentID),
    ->      FOREIGN KEY (CourseID) REFERENCES Courses_054(CourseID),
    ->      UNIQUE (StudentID, CourseID)
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> select * from Students_054;
+-----------+---------------+------+--------------+------+
| StudentID | Name          | Age  | DepartmentID | CGPA |
+-----------+---------------+------+--------------+------+
|         1 | Aarav Patel   |   20 |            1 | 9.10 |
|         2 | Sanya Mehta   |   21 |            1 | 8.50 |
|         3 | Rohan Gupta   |   22 |            2 | 7.80 |
|         4 | Isha Sharma   |   19 |            3 | 8.90 |
|         5 | Vikram Rao    |   23 |            2 | 6.70 |
|         6 | Ananya Kapoor |   21 |            1 | 9.50 |
|         7 | Kabir Khan    |   22 |            4 | 7.20 |
+-----------+---------------+------+--------------+------+
7 rows in set (0.00 sec)
```

```
mysql> select * from Enrollments_054;
+--------------+-----------+----------+-------+
| EnrollmentID | StudentID | CourseID | Marks |
+--------------+-----------+----------+-------+
|            1 |         1 | CSE101   |    92 |
|            2 |         1 | CSE102   |    88 |
|            3 |         2 | CSE101   |    75 |
|            4 |         2 | CSE102   |    81 |
|            5 |         3 | ECE201   |    67 |
|            6 |         4 | ME301    |    80 |
|            7 |         5 | ECE201   |    39 |
|            8 |         6 | CSE101   |    95 |
|            9 |         6 | CSE102   |    90 |
+--------------+-----------+----------+-------+
9 rows in set (0.00 sec)
```

SQL Queries:

1. Find students who have a CGPA higher than the university average CGPA

```
mysql> SELECT Name, CGPA
    -> FROM Students_054
    -> WHERE CGPA > (SELECT AVG(CGPA) FROM Students_054);
```

```
mysql> SELECT Name, CGPA
    -> FROM Students_054
    -> WHERE CGPA > (SELECT AVG(CGPA) FROM Students_054);
+---------------+------+
| Name          | CGPA |
+---------------+------+
| Aarav Patel   | 9.10 |
| Sanya Mehta   | 8.50 |
| Isha Sharma   | 8.90 |
| Ananya Kapoor | 9.50 |
+---------------+------+
4 rows in set (0.00 sec)
```

2. Find students who have never enrolled in any course

```
SELECT Name
    -> FROM Students_054
    -> WHERE StudentID NOT IN (SELECT StudentID FROM Enrollments_054);
```

```
mysql> SELECT Name
    -> FROM Students_054
    -> WHERE StudentID NOT IN (SELECT StudentID FROM Enrollments_054);
+------------+
| Name       |
+------------+
| Kabir Khan |
+------------+
1 row in set (0.00 sec)
```

3. Get courses that have no students enrolled

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID NOT IN (SELECT CourseID FROM Enrollments_054);
```

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID NOT IN (SELECT CourseID FROM Enrollments_054);
+---------------------+
| CourseName          |
+---------------------+
| Structural Analysis |
| Digital Electronics |
+---------------------+
2 rows in set (0.00 sec)
```

4. Get students enrolled in courses offered by the 'Computer Sci' department

```
mysql> SELECT DISTINCT S.Name
    -> FROM Students_054 S
    -> JOIN Enrollments_054 E ON S.StudentID = E.StudentID
    -> WHERE E.CourseID IN (
    ->     SELECT CourseID
    ->     FROM Courses_054
    ->     WHERE DepartmentID = (SELECT DepartmentID FROM Departments_054 WHERE DeptName = 'Computer Science')
    -> );
```

```
mysql> SELECT DISTINCT S.Name
    -> FROM Students_054 S
    -> JOIN Enrollments_054 E ON S.StudentID = E.StudentID
    -> WHERE E.CourseID IN (
    ->     SELECT CourseID
    ->     FROM Courses_054
    ->     WHERE DepartmentID = (SELECT DepartmentID FROM Departments_054 WHERE DeptName
= 'Computer Science')
    -> );
+---------------+
| Name          |
+---------------+
| Aarav Patel   |
| Sanya Mehta   |
| Ananya Kapoor |
+---------------+
3 rows in set (0.00 sec)
```

5. Find students who have the same age as the oldest student

SELECT Name, Age
-> FROM Students_054
-> WHERE Age = (SELECT MAX(Age) FROM Students_054);

```
mysql> SELECT Name, Age
    -> FROM Students_054
    -> WHERE Age = (SELECT MAX(Age) FROM Students_054);
+------------+------+
| Name       | Age  |
+------------+------+
| Vikram Rao |   23 |
+------------+------+
1 row in set (0.00 sec)
```

6. Retrieve courses where at least one student scored more than 85 marks

mysql> SELECT DISTINCT CourseName
-> FROM Courses_054
-> WHERE CourseID IN (SELECT CourseID FROM Enrollments_054 WHERE Marks > 85);

```
mysql> SELECT DISTINCT CourseName
    -> FROM Courses_054
    -> WHERE CourseID IN (SELECT CourseID FROM Enrollments_054 WHERE Marks > 85);
+-----------------------------+
| CourseName                  |
+-----------------------------+
| Introduction to Programming |
| Data Structures             |
+-----------------------------+
2 rows in set (0.00 sec)
```

7. Find the highest CGPA in each department

```
SELECT D.DeptName, MAX(S.CGPA) AS HighestCGPA
    -> FROM Students_054 S
    -> JOIN Departments_054 D ON S.DepartmentID = D.DepartmentID
    -> GROUP BY D.DeptName;
```

```
mysql> SELECT D.DeptName, MAX(S.CGPA) AS HighestCGPA
    -> FROM Students_054 S
    -> JOIN Departments_054 D ON S.DepartmentID = D.DepartmentID
    -> GROUP BY D.DeptName;
+------------------+-------------+
| DeptName         | HighestCGPA |
+------------------+-------------+
| Civil            |        7.20 |
| Computer Science |        9.50 |
| Electronics      |        7.80 |
| Mechanical       |        8.90 |
+------------------+-------------+
4 rows in set (0.00 sec)
```

8. Get students who have enrolled in more than one course

```
SELECT Name
    -> FROM Students_054
    -> WHERE StudentID IN (
    ->     SELECT StudentID
    ->     FROM Enrollments_054
    ->     GROUP BY StudentID
    ->     HAVING COUNT(CourseID) > 1
    -> );
```

```
mysql> SELECT Name
    -> FROM Students_054
    -> WHERE StudentID IN (
    ->     SELECT StudentID
    ->     FROM Enrollments_054
    ->     GROUP BY StudentID
    ->     HAVING COUNT(CourseID) > 1
    -> );
+---------------+
| Name          |
+---------------+
| Aarav Patel   |
| Sanya Mehta   |
| Ananya Kapoor |
+---------------+
3 rows in set (0.00 sec)
```

9. Get the department with the most students enrolled

```
mysql> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
    ->     GROUP BY DepartmentID
    ->     ORDER BY COUNT(StudentID) DESC
    ->     LIMIT 1
    -> );
```

```
mysql> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
    ->     GROUP BY DepartmentID
    ->     ORDER BY COUNT(StudentID) DESC
    ->     LIMIT 1
    -> );
+------------------+
| DeptName         |
+------------------+
| Computer Science |
+------------------+
1 row in set (0.00 sec)
```

10. Find students who are not enrolled in any course

```
mysql> SELECT Name
    -> FROM Students_054
    -> WHERE StudentID NOT IN (SELECT StudentID FROM Enrollments_054);
```

```
mysql> SELECT Name
    -> FROM Students_054
    -> WHERE StudentID NOT IN (SELECT StudentID FROM Enrollments_054);
+------------+
| Name       |
+------------+
| Kabir Khan |
+------------+
1 row in set (0.00 sec)
```

11. Retrieve students who scored below the average marks in their courses

```
mysql> SELECT S.Name, E.CourseID, E.Marks
;   -> FROM Enrollments_054 E
    -> JOIN Students_054 S ON E.StudentID = S.StudentID
    -> WHERE E.Marks < (SELECT AVG(Marks) FROM Enrollments_054)
```

```
mysql> SELECT S.Name, E.CourseID, E.Marks
    -> FROM Enrollments_054 E
    -> JOIN Students_054 S ON E.StudentID = S.StudentID
    -> WHERE E.Marks < (SELECT AVG(Marks) FROM Enrollments_054);
+-------------+----------+-------+
| Name        | CourseID | Marks |
+-------------+----------+-------+
| Sanya Mehta | CSE101   |    75 |
| Rohan Gupta | ECE201   |    67 |
| Vikram Rao  | ECE201   |    39 |
+-------------+----------+-------+
3 rows in set (0.00 sec)
```

12. Find the department where the highest CGPA student belongs

```
mysql> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
    ->     WHERE CGPA = (SELECT MAX(CGPA) FROM Students_054)
    -> );
```

```
mysql> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
    ->     WHERE CGPA = (SELECT MAX(CGPA) FROM Students_054)
    -> );
+------------------+
| DeptName         |
+------------------+
| Computer Science |
+------------------+
1 row in set (0.00 sec)
```

13. Find the department with the highest number of students enrolled

```
> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
```

```
    ->      GROUP BY DepartmentID
    ->      ORDER BY COUNT(StudentID) DESC
    ->      LIMIT 1
    -> );
```

```
mysql> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
    ->     GROUP BY DepartmentID
    ->     ORDER BY COUNT(StudentID) DESC
    ->     LIMIT 1
    -> );
+------------------+
| DeptName         |
+------------------+
| Computer Science |
+------------------+
1 row in set (0.00 sec)
```

14. Get courses where the lowest score is higher than 70

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID IN (
    ->     SELECT CourseID
    ->     FROM Enrollments_054
    ->     GROUP BY CourseID
    ->     HAVING MIN(Marks) > 70
    -> );
```

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID IN (
    ->     SELECT CourseID
    ->     FROM Enrollments_054
    ->     GROUP BY CourseID
    ->     HAVING MIN(Marks) > 70
    -> );
+----------------------------+
| CourseName                 |
+----------------------------+
| Introduction to Programming |
| Data Structures            |
| Thermodynamics             |
+----------------------------+
3 rows in set (0.00 sec)
```

15. Retrieve students who have the same CGPA as 'Alice'

```
mysql> SELECT Name, CGPA
    -> FROM Students_054
    -> WHERE CGPA = (SELECT CGPA FROM Students_054 WHERE Name = 'Ananya Kapoor');
```

```
mysql> SELECT Name, CGPA
    -> FROM Students_054
    -> WHERE CGPA = (SELECT CGPA FROM Students_054 WHERE Name = 'Ananya Kapoor');
+---------------+------+
| Name          | CGPA |
+---------------+------+
| Ananya Kapoor | 9.50 |
+---------------+------+
1 row in set (0.00 sec)
```

16. Get the course where the maximum number of students have enrolled

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID = (
    ->     SELECT CourseID
    ->     FROM Enrollments_054
    ->     GROUP BY CourseID
    ->     ORDER BY COUNT(StudentID) DESC
    ->     LIMIT 1
    -> );
```

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID = (
    ->     SELECT CourseID
    ->     FROM Enrollments_054
    ->     GROUP BY CourseID
    ->     ORDER BY COUNT(StudentID) DESC
    ->     LIMIT 1
    -> );
+-----------------+
| CourseName      |
+-----------------+
| Data Structures |
+-----------------+
1 row in set (0.00 sec)
```

17. Find students who are enrolled in more than two courses

```
mysql> SELECT Name
    -> FROM Students_054
    -> WHERE StudentID IN (
    ->     SELECT StudentID
    ->     FROM Enrollments_054
```

```
-> GROUP BY StudentID
-> HAVING COUNT(CourseID) > 2
-> );
```

```
mysql> SELECT Name
    -> FROM Students_054
    -> WHERE StudentID IN (
    ->     SELECT StudentID
    ->     FROM Enrollments_054
    ->     GROUP BY StudentID
    ->     HAVING COUNT(CourseID) > 2
    -> );
Empty set (0.00 sec)
```

18. Retrieve students who have scored the highest marks in any course(Hint: Use Join along with nested queries).

```
SELECT DISTINCT S.Name, E.CourseID, E.Marks
-> FROM Students_054 S
-> JOIN Enrollments_054 E ON S.StudentID = E.StudentID
-> WHERE E.Marks = (
->     SELECT MAX(Marks)
->     FROM Enrollments_054
->     WHERE CourseID = E.CourseID
-> );
```

```
mysql> SELECT DISTINCT S.Name, E.CourseID, E.Marks
    -> FROM Students_054 S
    -> JOIN Enrollments_054 E ON S.StudentID = E.StudentID
    -> WHERE E.Marks = (
    ->     SELECT MAX(Marks)
    ->     FROM Enrollments_054
    ->     WHERE CourseID = E.CourseID
    -> );
+---------------+----------+-------+
| Name          | CourseID | Marks |
+---------------+----------+-------+
| Rohan Gupta   | ECE201   |    67 |
| Isha Sharma   | ME301    |    80 |
| Ananya Kapoor | CSE101   |    95 |
| Ananya Kapoor | CSE102   |    90 |
+---------------+----------+-------+
4 rows in set (0.00 sec)
```

19. Get the department with the lowest average CGPA

```
SELECT DeptName
-> FROM Departments_054
-> WHERE DepartmentID = (
->     SELECT DepartmentID
```

```
    ->      FROM Students_054
    ->      GROUP BY DepartmentID
    ->      ORDER BY AVG(CGPA) ASC
    ->      LIMIT 1
    -> );
```

```
mysql> SELECT DeptName
    -> FROM Departments_054
    -> WHERE DepartmentID = (
    ->     SELECT DepartmentID
    ->     FROM Students_054
    ->     GROUP BY DepartmentID
    ->     ORDER BY AVG(CGPA) ASC
    ->     LIMIT 1
    -> );
+----------+
| DeptName |
+----------+
| Civil    |
+----------+
1 row in set (0.00 sec)
```

20. Find the course in which the maximum number of students have failed (Marks < 40)

```
SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID = (
    ->      SELECT CourseID
    ->      FROM Enrollments_054
    ->      WHERE Marks < 40
    ->      GROUP BY CourseID
    ->      ORDER BY COUNT(StudentID) DESC
    ->      LIMIT 1
    -> );
```

```
mysql> SELECT CourseName
    -> FROM Courses_054
    -> WHERE CourseID = (
    ->     SELECT CourseID
    ->     FROM Enrollments_054
    ->     WHERE Marks < 40
    ->     GROUP BY CourseID
    ->     ORDER BY COUNT(StudentID) DESC
    ->     LIMIT 1
    -> );
+------------+
| CourseName |
+------------+
| Circuits   |
+------------+
```